Internship Task

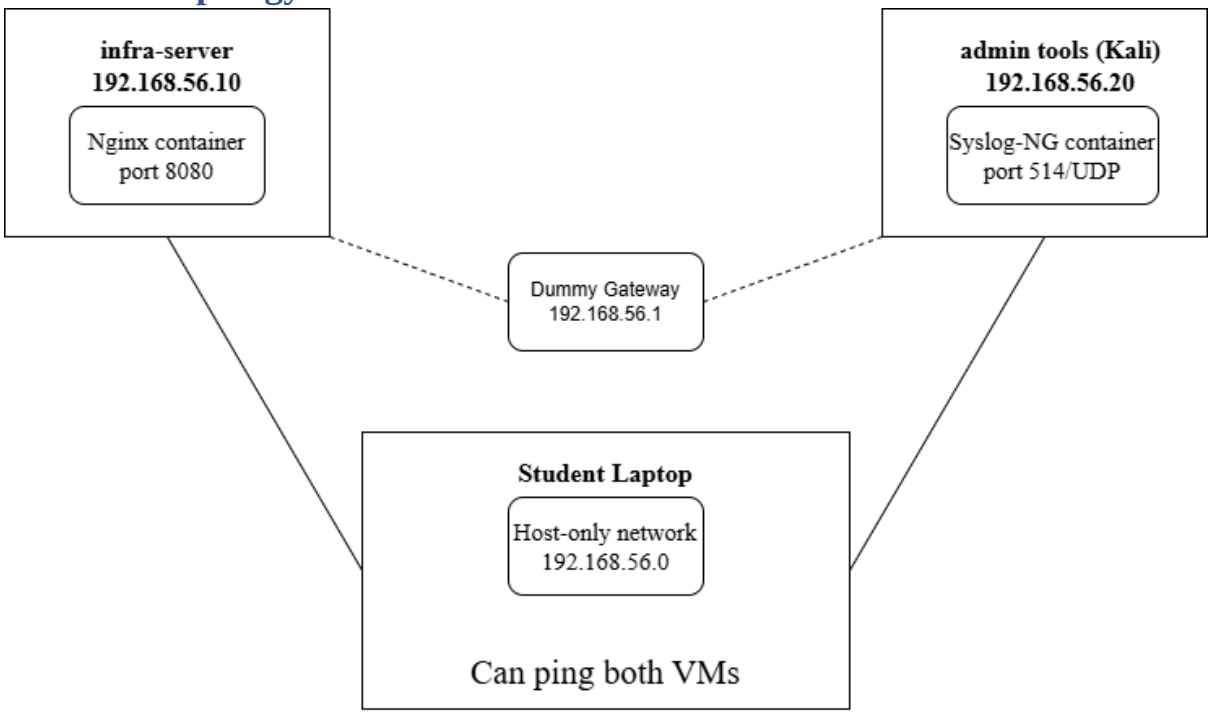
Week 2

Virtualization, Containerization & Python Scripting

Submitted by: Rania Muskan Malik
Date: 7/20/2026 1:30:22 AM

Table of Contents

Network Topology



IP/Port Table

Device	IP Address	Port	Purpose
infra-server	192.168.56.10	8080 (TCP)	Nginx container
admin-tools (Kali)	192.168.56.20	514 (UDP)	Syslog-NG container
Dummy Gateway	192.168.56.1	-	Not routable
Host	192.168.56.0	-	

Task 1: Virtualization

Ubuntu 22.04 was built in Virtual Box and an existing Kali VM was used. On both of them Adapter 1 from the network settings was set to host-only and a static ip address was applied.

```
rania@infra-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 10
0
    link/ether 08:00:27:6f:d7:04 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global enp0s3
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe6f:d704/64 scope link
       valid_lft forever preferred_lft forever
```

(infra-server showing a static ip of 192.168.56.10)

```
┌──(rania㉿kali)-[~/Desktop]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:57:c9:dc brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.20/24 brd 192.168.56.255 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe57:c9dc/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 2e:ff:53:02:bf:33 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
```

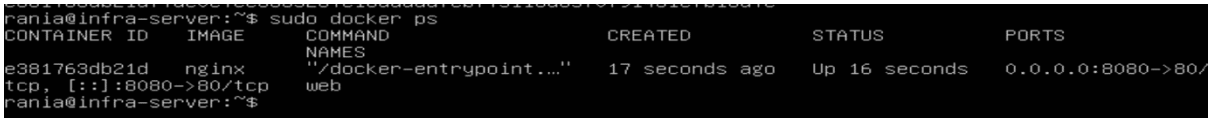
(kali showing a static ip of 192.168.56.20)

```
┌──(rania㉿kali)-[~/Desktop]
└─$ ping 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
64 bytes from 192.168.56.10: icmp_seq=1 ttl=64 time=1.58 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=64 time=7.41 ms
64 bytes from 192.168.56.10: icmp_seq=3 ttl=64 time=10.3 ms
64 bytes from 192.168.56.10: icmp_seq=4 ttl=64 time=3.57 ms
64 bytes from 192.168.56.10: icmp_seq=5 ttl=64 time=3.85 ms
64 bytes from 192.168.56.10: icmp_seq=6 ttl=64 time=35.4 ms
64 bytes from 192.168.56.10: icmp_seq=7 ttl=64 time=4.52 ms
64 bytes from 192.168.56.10: icmp_seq=8 ttl=64 time=9.11 ms
64 bytes from 192.168.56.10: icmp_seq=9 ttl=64 time=5.66 ms
64 bytes from 192.168.56.10: icmp_seq=10 ttl=64 time=6.42 ms
64 bytes from 192.168.56.10: icmp_seq=11 ttl=64 time=6.46 ms
64 bytes from 192.168.56.10: icmp_seq=12 ttl=64 time=16.4 ms
64 bytes from 192.168.56.10: icmp_seq=13 ttl=64 time=6.34 ms
64 bytes from 192.168.56.10: icmp_seq=14 ttl=64 time=4.58 ms
64 bytes from 192.168.56.10: icmp_seq=15 ttl=64 time=3.29 ms
64 bytes from 192.168.56.10: icmp_seq=16 ttl=64 time=4.49 ms
64 bytes from 192.168.56.10: icmp_seq=17 ttl=64 time=4.68 ms
64 bytes from 192.168.56.10: icmp_seq=18 ttl=64 time=2.29 ms
64 bytes from 192.168.56.10: icmp_seq=19 ttl=64 time=2.67 ms
64 bytes from 192.168.56.10: icmp_seq=20 ttl=64 time=3.88 ms
64 bytes from 192.168.56.10: icmp_seq=21 ttl=64 time=1.03 ms
64 bytes from 192.168.56.10: icmp_seq=22 ttl=64 time=2.29 ms
64 bytes from 192.168.56.10: icmp_seq=23 ttl=64 time=2.16 ms
64 bytes from 192.168.56.10: icmp_seq=24 ttl=64 time=3.79 ms
64 bytes from 192.168.56.10: icmp_seq=25 ttl=64 time=42.3 ms
```

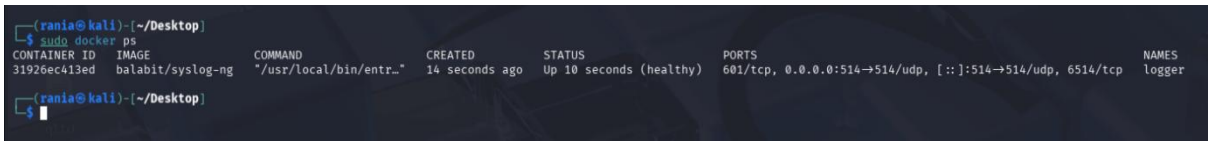
(from the kali vm a successful ping to the infra-server)

Task 2: Containerization

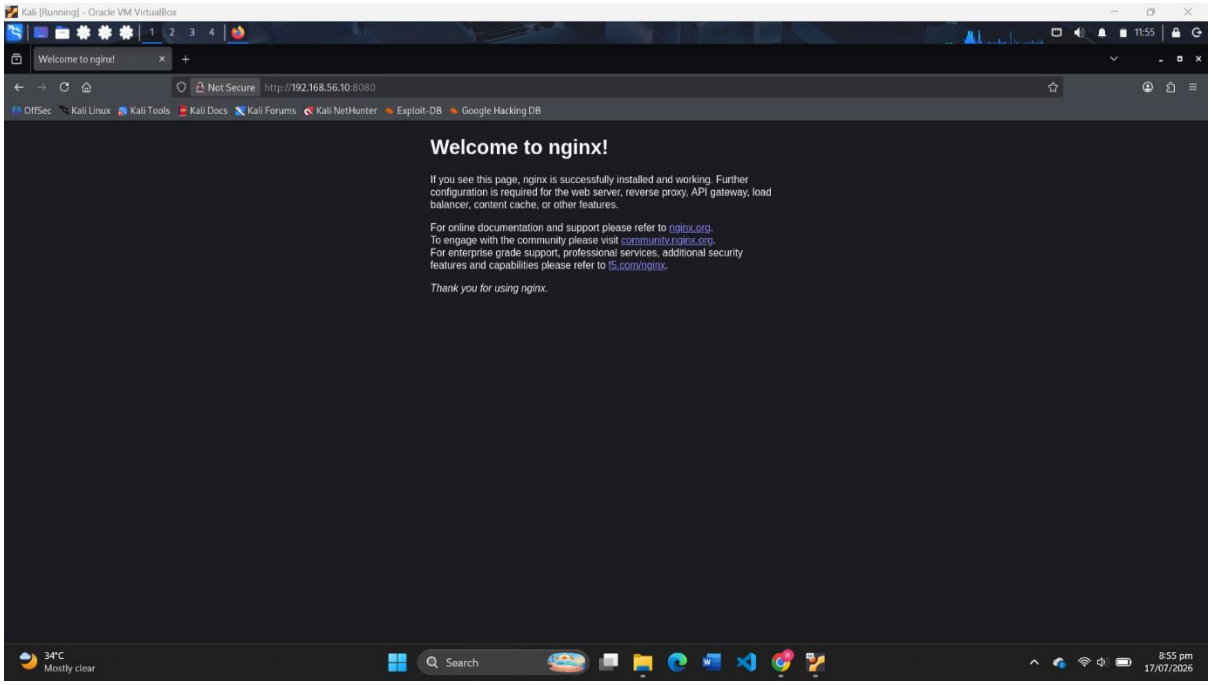
Docker was installed on both VMs. infra-server runs an Nginx container on a bridge network (web-net), exposed on host port 8080. admin-tools runs a Syslog-NG container on a separate bridge network (log-net), exposed on UDP port 514.



(docker ps on infra-server)



(docker ps on kali)

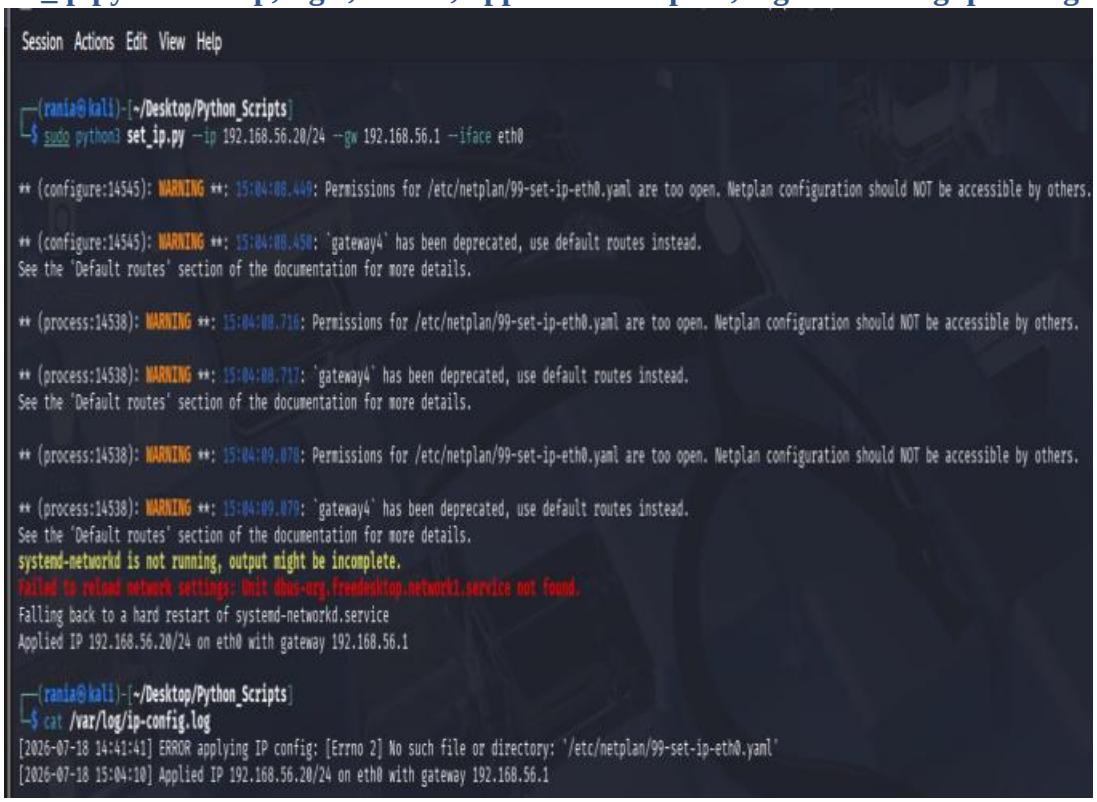


http://192.168.56.10:8080 (from admin-tools)

Syslog capture (logger "test" → view in container logs)

Task 3: Python Automation

set_ip.py: takes --ip, --gw, --iface; applies via Netplan; logs to /var/log/ip-config.log.



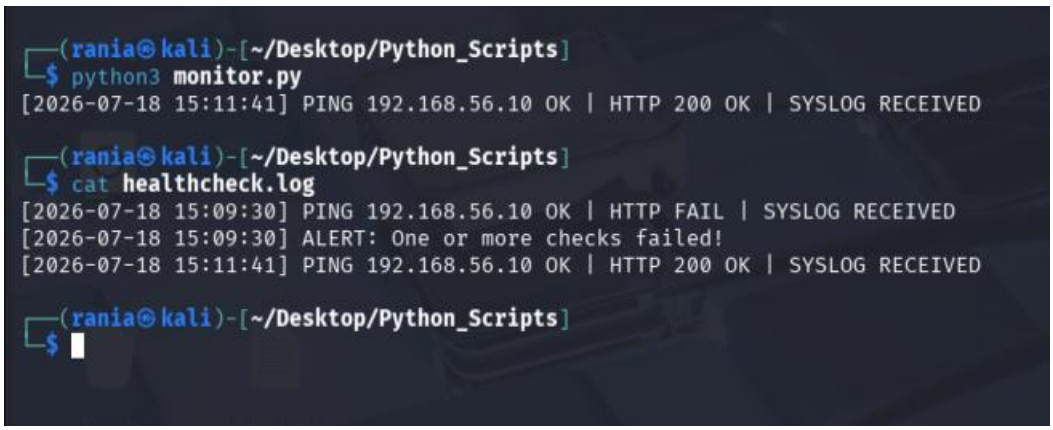
(set_ip.py running)

Explanation: The script automates the manual task which was being done in Task 1.

Real-world scenario: When you have to apply static ip addresses over a large number of machines, you will not prefer to do it by hand. You can just run the script and get the static ip addresses

monitor.py

1. Ping 192.168.56.10 (Nginx host)
2. HTTP-check Nginx: GET / to http://192.168.56.10:8080 (use requests lib)
3. Syslog test: run logger "Health test $(date)" → ensure message appears in Syslog-NG container (docker logs logger)
4. Log status lines to healthcheck.log with timestamps



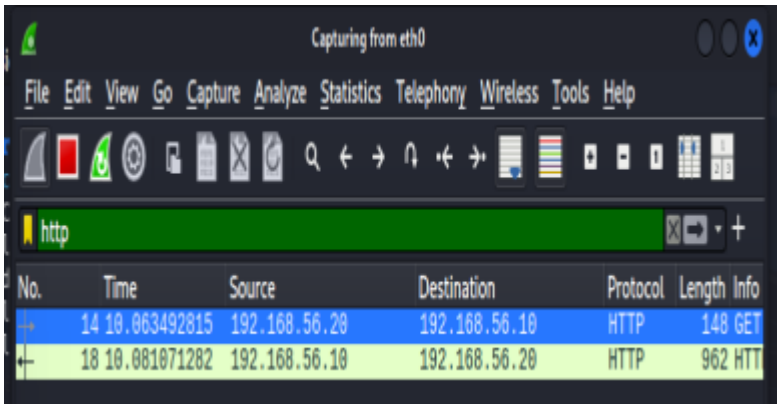
(monitor.py running)

Explanation: The script checks whether the infra-server can be pinged, can we get the web server response and whether logging is working

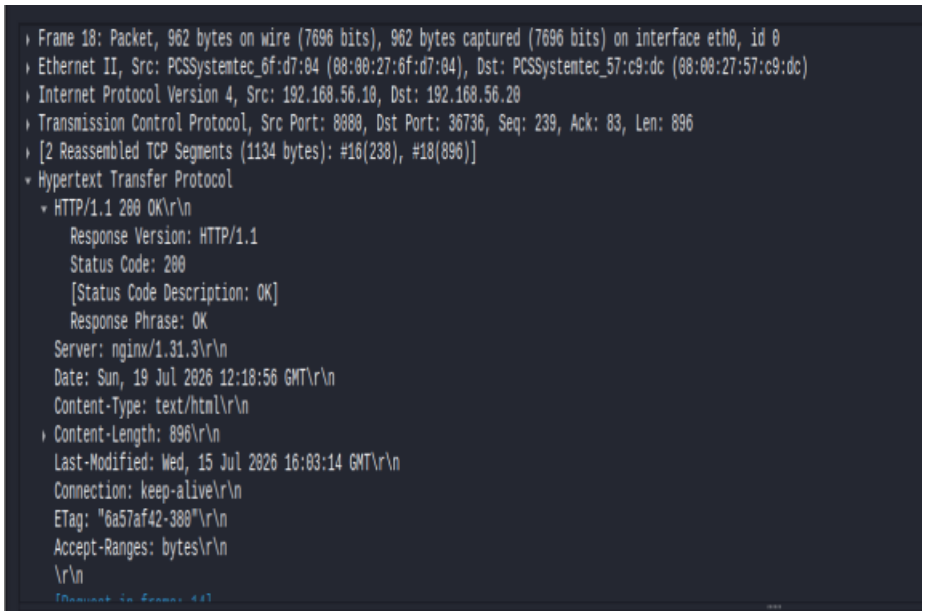
Real-world scenario: This basically resembles a tiny monitoring tool

Bonus: Packet Analysis Section

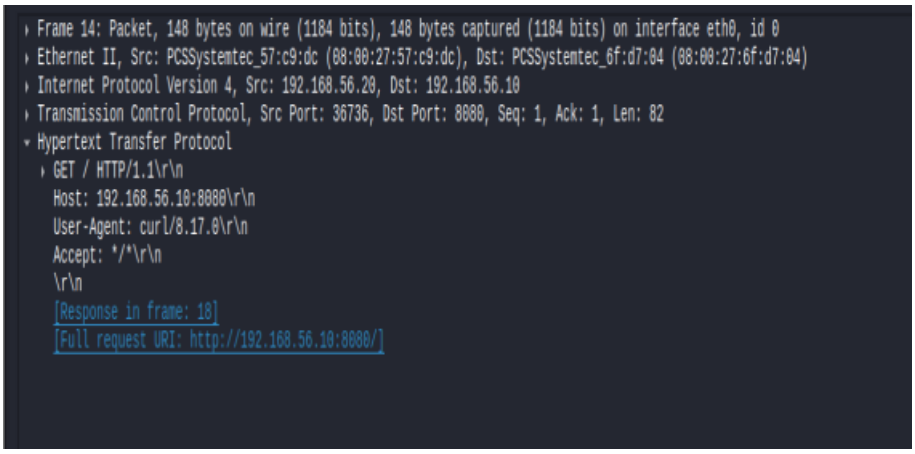
1. HTTP traffic (admin-tools → Nginx) – screenshot + header analysis



(Packet capture on wirshark shows http traffic)

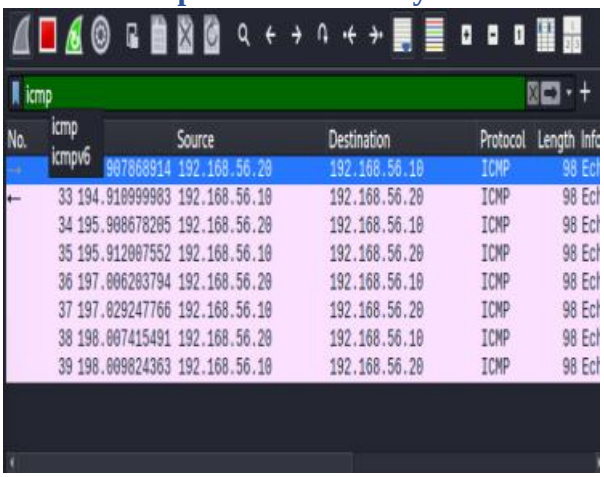


(http GET)

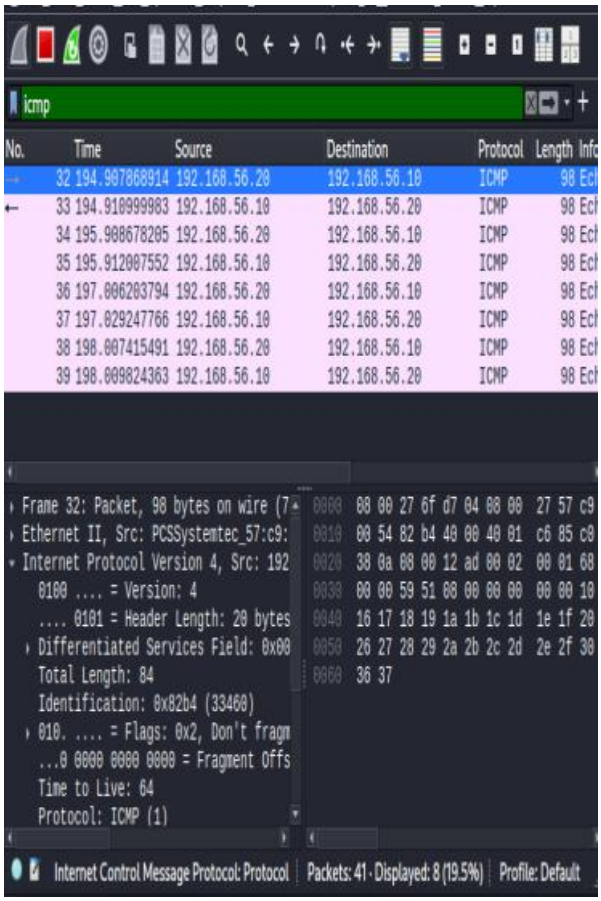


(http 200OK)

2. ICMP echo between VMs – prove connectivity & TTL values

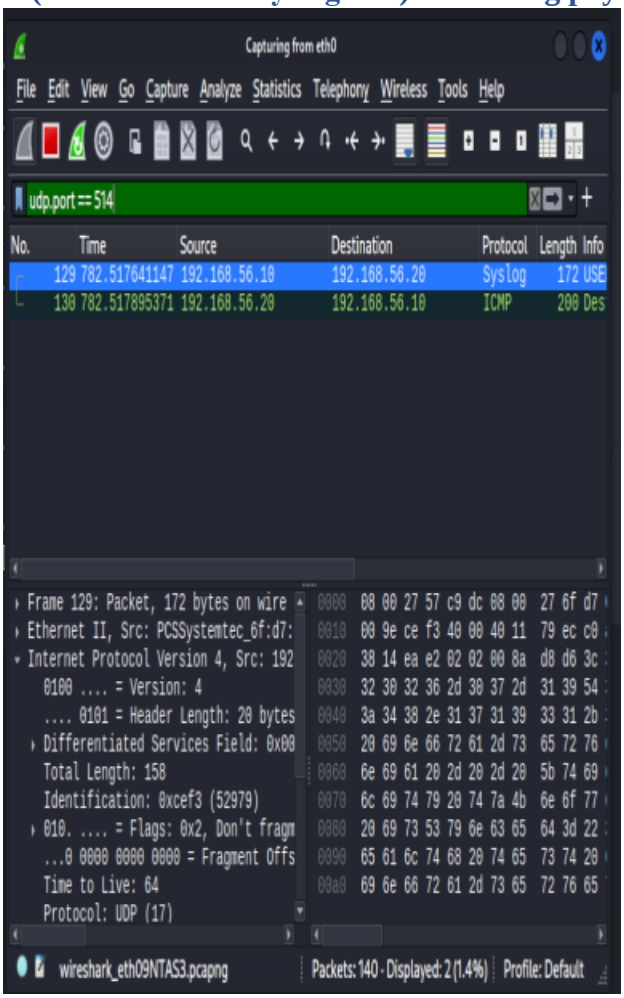


(Packet capture on wireshark shows icmp traffic)



(ICMP packet detail showing Time To Live (TTL) = 64, confirming direct connectivity)

3. Syslog UDP 514 (infra-server → Syslog-NG) – show log payload

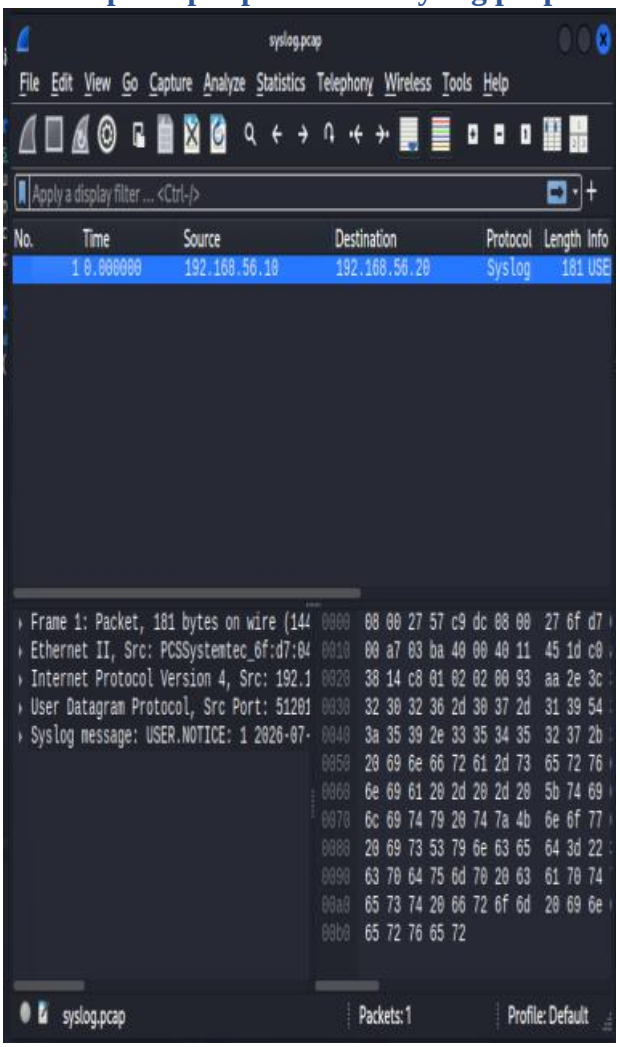


(Wireshark capture filtered on udp.port == 514, showing the syslog payload sent from infra-server to admin-tools)

4. Security note: plaintext HTTP vs HTTPS

For the working of this lab all the traffic of this lab is sent over http. http traffic is not encrypted so all the headers even can be viewed in plain-text. Now if the same was to be done using https the traffic would be encrypted using TLS. In a real-world scenario traffic is not communicated using http but rather with https

5. **Extra: capture with tcpdump -i port 514 -w syslog.pcap and open in Wireshark**



Reflection: What I Learned This Week

1. This week I learnt that when we are sending traffic over one host-only adapter then we need to up another adapter on NAT to access the internet
2. A gateway ip was posing default gateway conflicts during the whole work on kali and that had to removed manually again and again